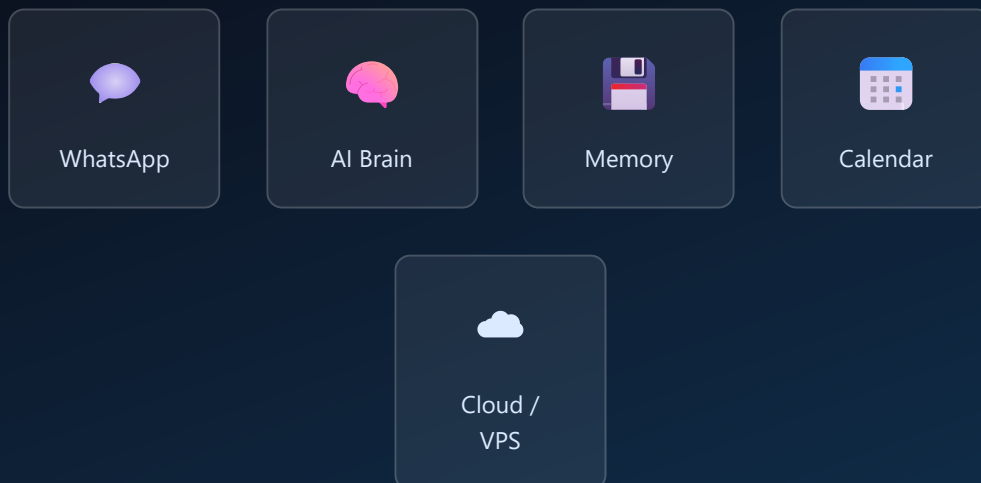


COMPLETE PROJECT DOCUMENTATION

# WhatsApp AI Agent

From WhatsApp message to AI conversation, memory, calendar  
booking,  
and production deployment.



Generated from the actual project source code · C:\Websites\WhatsApp AI Agent

# Table of Contents

---

## Part 1 — Understanding the Project

|                                              |    |
|----------------------------------------------|----|
| 1. Executive Summary — What Is This Project? | 5  |
| 2. Complete System Architecture              | 7  |
| 3. Every Connection Explained (A–L)          | 10 |
| 4. Message Lifecycle (Step by Step)          | 14 |
| 5. Real Conversation Examples                | 16 |

## Part 2 — How It Works Inside

|                                                     |    |
|-----------------------------------------------------|----|
| 6. Security                                         | 19 |
| 7. Error Handling                                   | 21 |
| 8. The Database (SQLite Memory)                     | 22 |
| 9. The AI Agent Loop                                | 24 |
| 10. The Booking System                              | 26 |
| 11. Configuration Reference (Environment Variables) | 28 |
| 12. Testing                                         | 31 |

## Part 3 — Running & Deploying

|                                        |    |
|----------------------------------------|----|
| 13. Running It Locally                 | 34 |
| 14. Deploying to a VPS                 | 35 |
| 15. Setting Up Meta WhatsApp Cloud API | 37 |
| 16. Setting Up OpenRouter              | 38 |
| 17. Setting Up Google Calendar         | 39 |
| 18. Managing Business Knowledge        | 40 |
| 19. Project File & Folder Map          | 41 |
| 20. Monitoring & Maintenance           | 42 |

## Part 4 — Status & Reference

|                                           |    |
|-------------------------------------------|----|
| 21. Current Project Status Dashboard      | 44 |
| 22. Limitations & Risks                   | 45 |
| 23. Explain This Project to a 10-Year-Old | 46 |
| 24. Glossary                              | 48 |

25. Final Master Diagram

50

26. Source of Truth & Generation Record

52

## PART 1

# Understanding the Project

What this system is, how the pieces connect, and what happens from the moment a customer types a message.

# 1. Executive Summary

---

## What is this project?

The **WhatsApp AI Agent** is a computer program that lets a business talk to its customers on WhatsApp automatically. A customer sends a normal WhatsApp message, and this program reads it, thinks about it using an AI model, remembers earlier parts of the conversation, answers questions using the business's own information, and — if the customer wants — checks a real calendar and books a real appointment. All of this happens without a human typing the reply.

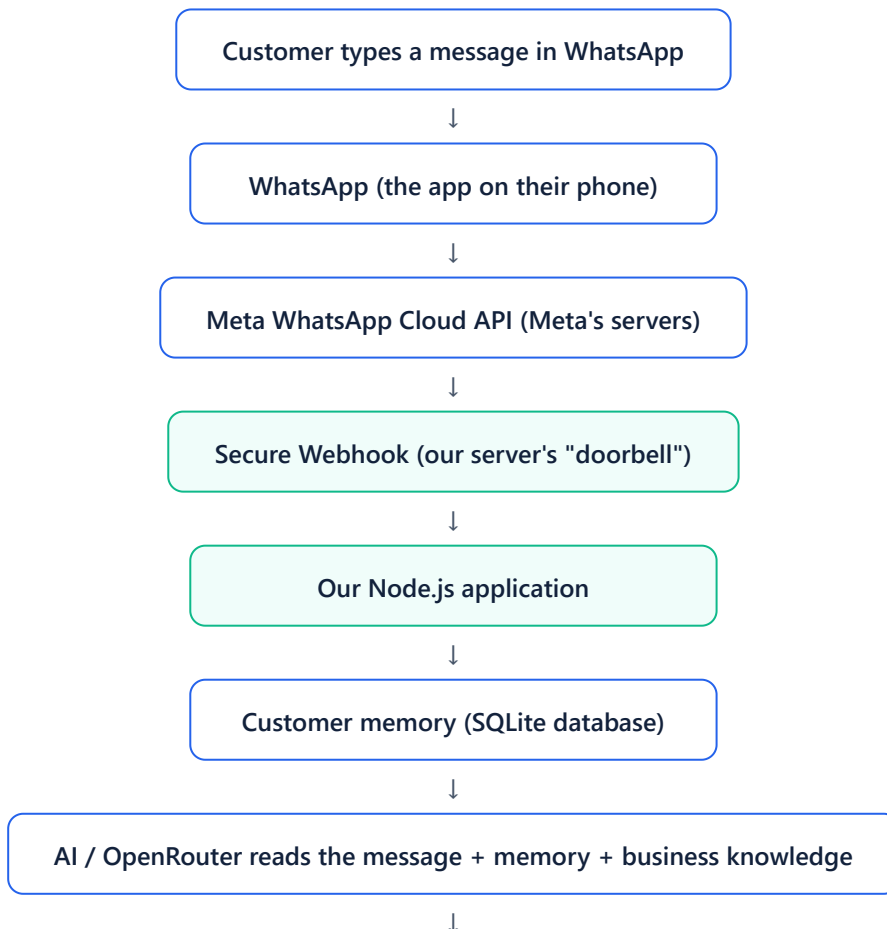
## What problem does it solve?

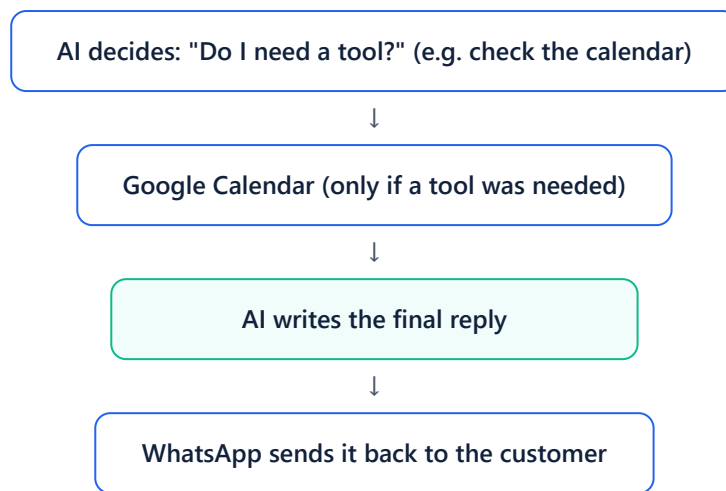
Many small businesses cannot answer every WhatsApp message instantly, especially outside office hours. Customers ask the same questions repeatedly ("what do you charge?", "are you open Saturday?", "can I book Tuesday at 3?"). This project answers those automatically, using only facts the business owner has approved, and hands off to a human when it doesn't know something.

## What happens when a customer sends a message?

### EXAMPLE

Customer sends: *"Hello, I want to book an appointment tomorrow."*





## The Smart Receptionist analogy

### EXPLAIN LIKE I'M 10

Imagine this whole system is **a smart receptionist working at a front desk**:

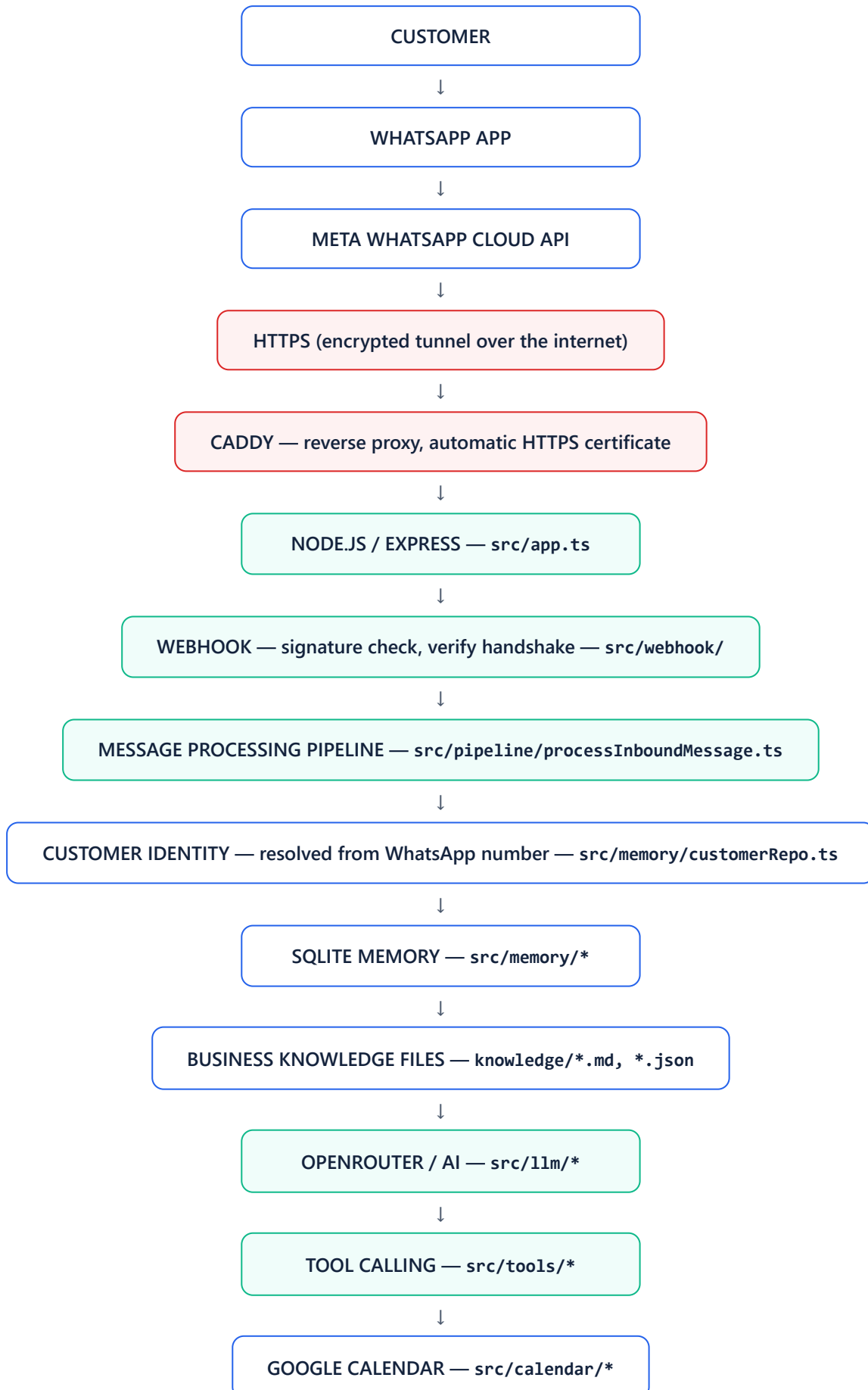
- **WhatsApp** = the telephone the customer calls on
- **AI (OpenRouter)** = the receptionist's brain, doing the thinking
- **SQLite** = the notebook where the receptionist writes down who called and what was said
- **Google Calendar** = the appointment diary on the desk
- **Webhook** = the doorbell that tells the receptionist "someone is here!"
- **Knowledge files** = the instruction binder listing prices, services and rules
- **Docker** = the box that lets the whole desk be moved to any building and still work the same way
- **Caddy** = the security guard at the door who checks IDs (HTTPS) before anyone gets in

### FOR THE TECHNICAL READER

This is a Node.js/TypeScript Express server that receives Meta WhatsApp Cloud API webhook events, resolves a customer by their WhatsApp number, loads/persists conversation state in SQLite, builds an OpenRouter chat-completion request with a tool-calling loop (`check_availability`, `book_appointment`), and calls the Google Calendar API (service account auth) for real availability and booking. It runs as a Docker container behind Caddy for automatic HTTPS, intended for a single VPS.

## 2. Complete System Architecture

This diagram matches the real source code layout in `src/`. Every box below corresponds to actual files, not a plan.





## WHATSAPP RESPONSE — `src/whatsapp/client.ts`

### Cross-cutting concerns (present at every step)

| Concern                               | Where it lives                                                             | What it does                                                                                                                 |
|---------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Webhook signature verification        | <code>src/webhook/verifySignature.ts</code>                                | Confirms the request really came from Meta (HMAC-SHA256), not an impostor.                                                   |
| Idempotency / duplicate protection    | <code>src/memory/webhookEventRepo.ts</code>                                | Meta's message ID is stored once; a repeat delivery is silently ignored.                                                     |
| Per-customer concurrency lock         | <code>src/pipeline/idempotency.ts</code> (with <code>CustomerLock</code> ) | Two messages from the same customer never get processed at the same time.                                                    |
| Retry / timeout handling              | <code>src/utils/retry.ts</code>                                            | Retries WhatsApp/OpenRouter/Calendar calls with backoff on transient errors; gives up immediately on permanent (4xx) errors. |
| Structured logging + secret redaction | <code>src/logger.ts</code>                                                 | Every log line is JSON; tokens/keys/signatures are automatically redacted; WhatsApp numbers are masked.                      |
| Rate limiting                         | <code>src/app.ts</code> (express-rate-limit)                               | Caps requests per minute per IP on <code>/webhook</code> .                                                                   |
| Security headers (Helmet)             | <code>src/app.ts</code>                                                    | Standard hardening headers on every response.                                                                                |
| Health endpoint                       | <code>src/health/router.ts</code> — GET <code>/health</code>               | "Is the process alive?" — used by Docker's HEALTHCHECK.                                                                      |
| Readiness endpoint                    | <code>src/health/router.ts</code> — GET <code>/readiness</code>            | "Can it actually reach the database?"                                                                                        |
| Graceful shutdown                     | <code>src/index.ts</code>                                                  | On SIGTERM/SIGINT, stops accepting new requests, closes the DB, then exits (forced after 10s if stuck).                      |

### Simplified architecture for a beginner

#### SIMPLE VERSION

Customer sends a WhatsApp message



Our program reads it and remembers who is asking



The AI brain thinks about the best answer, checking the calendar if needed



The answer is sent back on WhatsApp

### 3. Every Connection Explained

#### A. Customer → WhatsApp

The customer opens WhatsApp on their phone and types "Hi". WhatsApp packages this as a message and sends it to Meta's servers over the internet — nothing our code does yet; this step happens entirely inside the WhatsApp app and Meta's network.

#### B. WhatsApp → Meta Cloud API

Meta owns WhatsApp and runs the "Cloud API" — a set of servers that receive every message sent to a business's WhatsApp number and can send messages back on the business's behalf. Our application never talks to the customer's phone directly; it always talks to Meta, and Meta relays the message.

#### C. Meta → Webhook (POST /webhook)

**WHAT IS A WEBHOOK?**

A webhook is like a doorbell. Instead of our program constantly walking to the door and asking "did anyone arrive yet?" (which wastes effort), we give Meta our doorbell number (a URL) and Meta rings it themselves the instant a new message shows up. Our code (`src/webhook/router.ts`) is always listening for that ring.

Real endpoint in this project: POST `https://<your-domain>/webhook`, implemented in `src/webhook/router.ts`.

#### D. Webhook Security

| Layer                                 | Purpose                                                                                                                                                                                                      | Source file                                 |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| GET verification (one-time handshake) | When you first register the webhook URL with Meta, Meta sends a GET request with a secret <code>hub.verify_token</code> . We check it matches our own <code>WHATSAPP_VERIFY_TOKEN</code> before confirming.  | <code>src/webhook/verifyToken.ts</code>     |
| POST signature (every single message) | Meta signs every webhook POST with <code>X-Hub-Signature-256</code> , computed from the exact request body and our <code>META_APP_SECRET</code> . We recompute the same signature and compare.               | <code>src/webhook/verifySignature.ts</code> |
| Timing-safe comparison                | We compare signatures with <code>crypto.timingSafeEqual</code> , not <code>===</code> , so an attacker can't guess the correct signature one character at a time by measuring how fast the comparison fails. | same file                                   |

**WHY THIS MATTERS**

Without this check, anyone on the internet could send a fake message to our webhook pretending to be a real customer — for example, pretending someone confirmed a booking they never made. The signature

proves the message truly came from Meta.

## E. Node.js / Express — the central controller

Express is the framework that receives every HTTP request and decides which piece of code should handle it. In this project, it's the single front door: it applies security headers (Helmet), logging, rate limiting, and JSON parsing, then routes `/health`, `/readiness`, and `/webhook` to their handlers. It's "central" because every request — and only requests that came in this way — reach the rest of the system.

## F. SQLite — where the receptionist writes things down

SQLite is a database that lives in a single file (`data/app.db`) rather than a separate server. This project stores:

- **customers** — one row per WhatsApp number that has ever messaged
- **conversations** — a "session" of memory for a customer; a new one starts on restart
- **conversation\_messages** — every message, in order, so the AI has real context
- **booking\_sessions** — partial booking details being collected across turns
- **webhook\_events** — Meta's message IDs, so the same message is never processed twice

See Chapter 8 for the full schema diagram and an explanation of every table.

## G. Knowledge files — the instruction binder

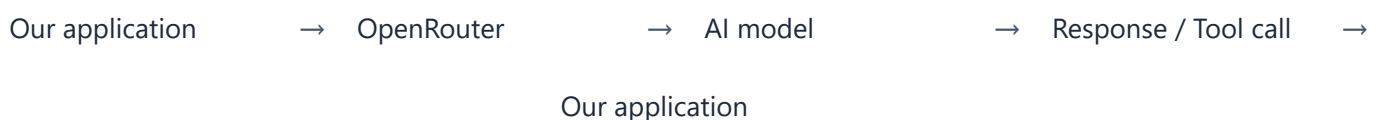
```
knowledge/  
├── business.md  
├── services.md  
├── policies.md  
├── faq.md  
└── booking.json
```

These are plain Markdown/JSON files, loaded once at startup by `src/knowledge/loader.ts` and pasted into the AI's system prompt. Business facts (prices, hours, policies) live here — **not** hard-coded in the TypeScript source — so a non-developer can edit a text file to change what the AI says, without touching code. The system prompt (`src/llm/buildSystemPrompt.ts`) explicitly instructs the AI to answer only from these files and never invent facts.

### PLANNED / NOT YET CONFIGURED

The knowledge files shipped in this repository contain example/placeholder content only (clearly marked as such inside each file). They must be replaced with the real business's information before going live — see Chapter 18.

## H. OpenRouter — the connection to the AI model



OpenRouter is a single API that can talk to many different AI models. The model actually used is set by the `OPENROUTER_MODEL` environment variable (`src/config/env.ts` defaults it to `openai/gpt-4o-mini` if not overridden) — changing models means changing one environment variable, not rewriting code.

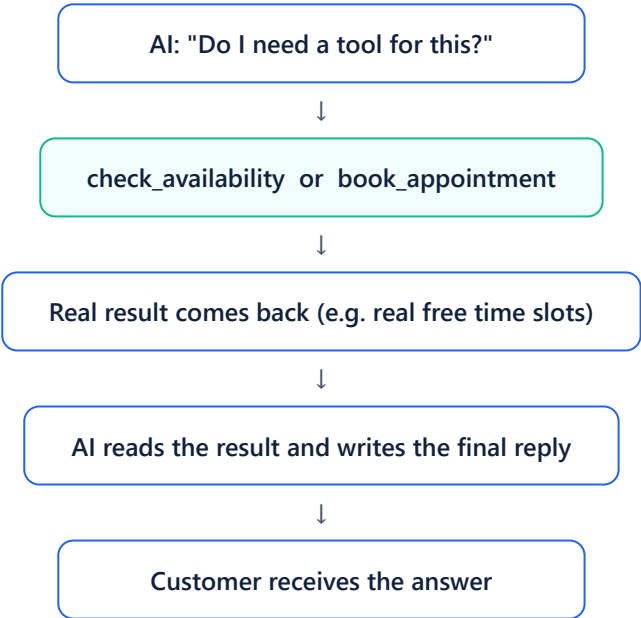
**NOTE**

This documentation does not claim a specific model is "live" in production — that depends entirely on what value is placed in the deployed `.env` file for `OPENROUTER_MODEL`.

## I. Tool calling — the receptionist's special abilities

**RECEPTIONIST ANALOGY**

A good receptionist doesn't just talk — sometimes they need to walk over to the appointment diary and actually check it. "Tool calling" is the AI doing exactly that: it can pause mid-thought and say "I need to actually look this up," run a real function in our code, get a real answer back, and then continue.



Implemented in `src/tools/checkAvailability.ts`, `src/tools/bookAppointment.ts`, registered in `src/tools/index.ts`, and driven by the loop in `src/llm/agentLoop.ts`.

## J. Google Calendar



**Why check availability twice?** Between the moment the AI shows a customer a free slot and the moment they say "yes, book it," a few seconds (or longer) pass. In that window, someone else could book that exact slot.

src/calendar/booking.ts re-queries Google Calendar's free/busy data immediately before creating the event, and only proceeds if the slot is still free — this closes that race window instead of trusting a stale answer.

## K. Docker

### EXPLAIN LIKE I'M 10

Imagine packing the entire application — the code, the exact version of Node.js it needs, all its tools — into one sealed box. Wherever you ship that box (any computer, any VPS), it opens up and runs *exactly* the same way. That box is called a **container**, and Docker is the tool that builds and runs it.

| File                        | Purpose                                                                                                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dockerfile                  | Recipe for building the application "box": compiles the TypeScript, installs only production dependencies, runs as a non-root user, includes a healthcheck. |
| docker-compose.yml          | Runs two boxes together: the app, and Caddy (the HTTPS security guard) in front of it.                                                                      |
| Volumes (./data, ./secrets) | Folders on the real VPS disk that are shared into the container, so the SQLite database survives even if the container is rebuilt or restarted.             |

## L. Caddy — the security guard at the door

Internet → HTTPS → Caddy → Our application

Caddy is a web server placed in front of the application. Its job in docker-compose.yml/nginx/Caddyfile is to automatically obtain and renew a free HTTPS certificate (via Let's Encrypt) for the domain configured in DOMAIN, and forward traffic to the app container over plain HTTP internally. Meta refuses to send webhooks to a non-HTTPS URL, so this step is required before the webhook can be registered.

# 4. Message Lifecycle — Step by Step

Every single inbound WhatsApp message goes through exactly this sequence (source: `src/webhook/router.ts` + `src/pipeline/processInboundMessage.ts`):

| #  | Step                                         | In simple words                                                                                           |
|----|----------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 1  | Customer sends a message                     | They type in WhatsApp and hit send.                                                                       |
| 2  | Meta receives it                             | Meta's servers get the message first.                                                                     |
| 3  | Meta calls our webhook                       | Meta rings our doorbell with the message attached.                                                        |
| 4  | Server verifies the request                  | We check the signature is really from Meta.                                                               |
| 5  | Server replies 200 OK immediately            | We tell Meta "got it" right away, before doing any AI work, so Meta doesn't think we're broken and retry. |
| 6  | Message ID is checked                        | Have we seen this exact message before?                                                                   |
| 7  | Duplicate messages are ignored               | If yes — stop here, do nothing else.                                                                      |
| 8  | Customer is identified by <code>wa_id</code> | Their WhatsApp number becomes their ID automatically.                                                     |
| 9  | Customer lock is acquired                    | Make sure no other message from this same customer runs at the same time.                                 |
| 10 | Conversation is loaded                       | Pull up their notebook page from SQLite.                                                                  |
| 11 | Restart command checked                      | Did they just say "restart" or "reset"?                                                                   |
| 12 | Knowledge is used                            | Business facts are added to what the AI can see.                                                          |
| 13 | AI receives full context                     | System instructions + history + knowledge, sent to OpenRouter.                                            |
| 14 | AI answers OR requests a tool                | Either a plain reply, or "let me check the calendar first."                                               |
| 15 | Tool executes                                | Our code actually calls Google Calendar (if needed).                                                      |
| 16 | Tool result goes back to the AI              | The AI sees the real answer, not a guess.                                                                 |
| 17 | AI generates the final response              | Now it writes the message the customer will actually see.                                                 |
| 18 | WhatsApp API sends the response              | Our code calls Meta's API to deliver the reply.                                                           |
| 19 | Conversation is stored                       | Every turn is saved to SQLite for next time.                                                              |
| 20 | Lock released                                | The next message from this customer (or anyone) can now be processed.                                     |

## SOURCE MAPPING

Steps 4–7 → `src/webhook/router.ts`. Steps 8–11 → `src/pipeline/processInboundMessage.ts` + `src/restart/isRestartCommand.ts`. Steps 12–17 → `src/llm/agentLoop.ts` + `src/tools/*`. Step 18 → `src/whatsapp/client.ts`. Steps 19–20 → `src/memory/conversationRepo.ts` + `src/pipeline/idempotency.ts`.

## 5. Real Conversation Examples

These examples reflect the actual behavior verified by the test suite (tests/integration/conversationFlow.e2e.test.ts), not hypothetical behavior.

### Example 1 — First contact

**Customer:** "Hi"

**Agent:** "Hello! How can I help you today?"

A brand-new customer is automatically created in the `customers` table using their WhatsApp number — no signup form, no manual step.

### Example 2 — Question answered from knowledge

**Customer:** "What services do you provide?"

**Agent:** answers using only what's written in `knowledge/services.md` — it cannot invent a service that isn't listed there.

### Example 3 — Checking availability

**Customer:** "I want an appointment tomorrow."

AI → `check_availability` → Google Calendar → real free slots →  
AI's reply

### Example 4 — Booking it

**Customer:** "Book it for me."

AI → `book_appointment` → `re-check` → create event →  
confirmation

The agent never says "you're booked" unless `book_appointment` actually returned success.

### Example 5 — Two messages arrive together

If the same customer sends two messages almost simultaneously (or Meta redelivers one while the first is still processing), `withCustomerLock` (`src/pipeline/idempotency.ts`) makes the second one wait until the first fully finishes — they are processed one at a time, in order, never in parallel. This is proven by the concurrency test in `tests/integration/conversationFlow.e2e.test.ts`, which fires three simultaneous messages and confirms all three land in the database without corruption.

## Example 6 — Duplicate delivery

If Meta delivers the exact same message ID twice (a normal retry behavior on their side), `src/memory/webhookEventRepo.ts`'s `INSERT OR IGNORE` means the second attempt is recognized as already-seen and silently skipped — no second AI call, no second reply.

## Example 7 — "restart"

**Customer:** "restart"

**Agent:** "Done — I've started a fresh conversation. How can I help you?"

The current conversation row is marked ended, any in-progress booking session is cleared, and a brand-new conversation begins — but nothing is deleted; the old conversation and its messages remain in the database for history/audit.

## Example 8 — AI or provider failure

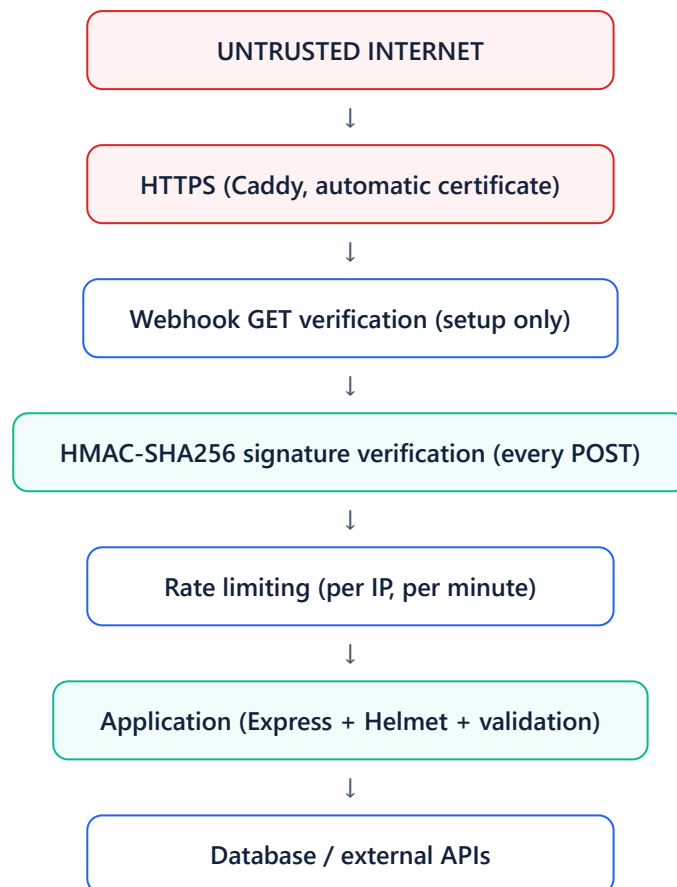
If OpenRouter (or any step in the agent loop) throws an error, the customer is never shown a stack trace or technical detail. They receive a plain, safe message ("Sorry, something went wrong on our end...") while the real error is logged internally for the developer to see.

## PART 2

# How It Works Inside

Security, error handling, the database, the AI loop, and the booking system — the internals a developer or maintainer needs.

## 6. Security



Every layer exists to stop a different kind of attacker: HTTPS stops eavesdropping, the signature stops impersonation, rate limiting stops flooding, and validation stops malformed/malicious payloads reaching business logic.

| Protection                      | Simple explanation                                           | Technical detail                                                                                                     |
|---------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| HTTPS                           | Locks the message in a tunnel only our server can open.      | Caddy auto-provisions a Let's Encrypt certificate for DOMAIN.                                                        |
| Webhook signature (HMAC-SHA256) | Proves a message really came from Meta.                      | src/webhook/verifySignature.ts — HMAC over the raw request body using META_APP_SECRET.                               |
| Timing-safe comparison          | Stops attackers guessing the signature bit by bit.           | crypto.timingSafeEqual instead of string equality.                                                                   |
| Rate limiting                   | Stops one source from flooding the doorbell.                 | express-rate-limit, RATE_LIMIT_PER_MINUTE (default 120/min) on /webhook.                                             |
| Helmet security headers         | Standard browser-facing hardening.                           | helmet() in src/app.ts; removes X-Powered-By, sets X-Content-Type-Options: nosniff, etc.                             |
| Request body size limit         | Stops someone sending a huge payload to overload the server. | MAX_BODY_SIZE (default 1mb) enforced by express.json().                                                              |
| Parameterized SQL               | Stops "SQL injection" tricks in messages.                    | All queries use better-sqlite3 prepared statements with bound parameters — user text is never concatenated into SQL. |

|                                           |                                                      |                                                                                                                                                         |
|-------------------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Secret redaction in logs                  | Keys and tokens never appear in log files.           | <code>src/logger.ts</code> pino redact config masks <code>Authorization</code> , signature headers, and any field named <code>token/key/secret</code> . |
| Masked WhatsApp numbers in logs           | Customer phone numbers aren't fully exposed in logs. | <code>maskWaId()</code> shows only the last 4 digits.                                                                                                   |
| Malformed JSON handling                   | A broken message doesn't crash the server.           | Central error handler returns <code>400 invalid_json</code> , not a raw 500.                                                                            |
| Generic error responses                   | Customers/attackers never see internal details.      | Any unhandled error returns a generic <code>500 internal_server_error</code> with no stack trace.                                                       |
| Environment variables / <code>.env</code> | Secrets never live in the code.                      | <code>.env</code> is gitignored; all secrets loaded via <code>process.env</code> , validated at startup by <code>src/config/env.ts</code> .             |

## 7. Error Handling

This table shows the **actual** behavior implemented in the source code for each failure — nothing here is aspirational.

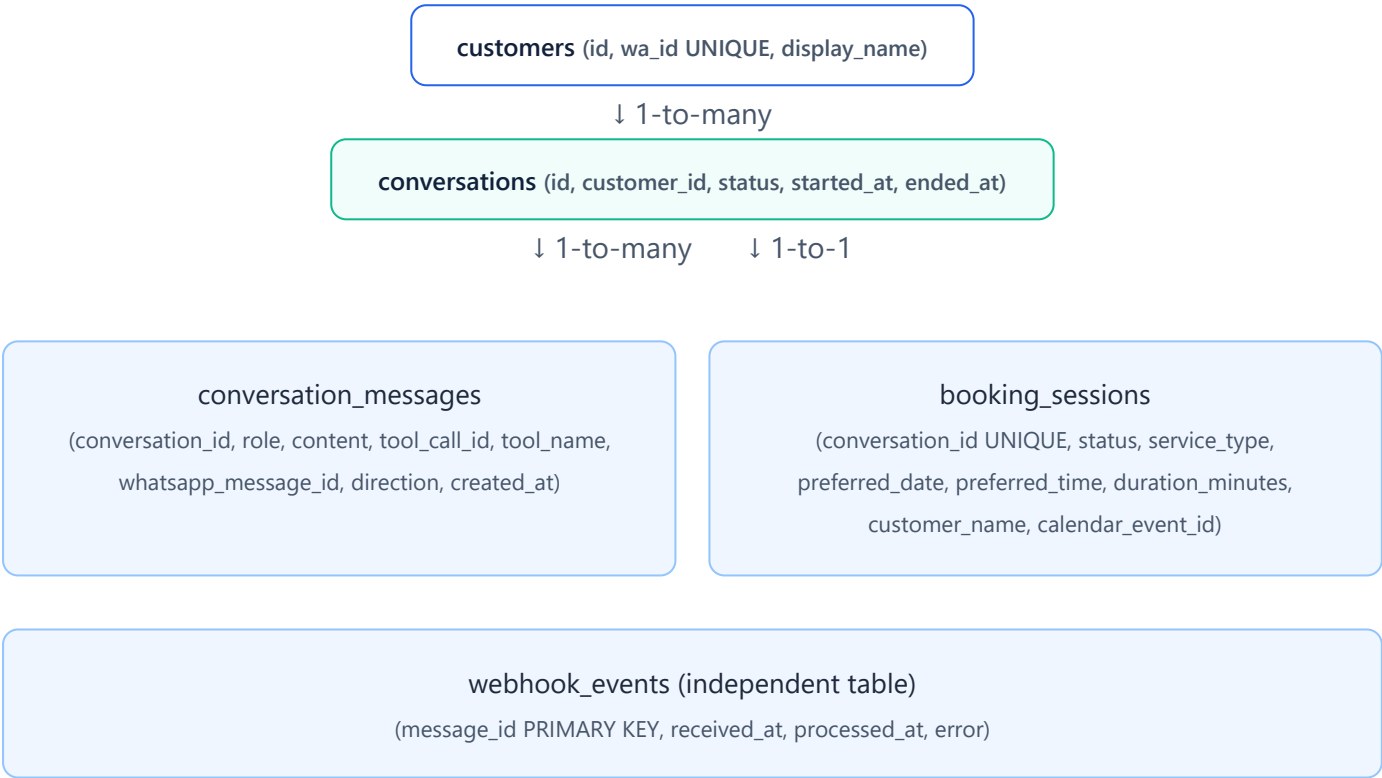
| Situation                           | What actually happens                                                                                                                                                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Malformed JSON body                 | <code>src/app.ts</code> central error handler catches <code>entity.parse.failed</code> and returns 400 <code>{"error": "invalid_json"}</code> .                                                                                                          |
| Oversized request body              | Returns 413 <code>{"error": "payload_too_large"}</code> (over <code>MAX_BODY_SIZE</code> ).                                                                                                                                                              |
| Invalid webhook signature           | <code>src/webhook/verifySignature.ts</code> returns 401 before any processing happens.                                                                                                                                                                   |
| Invalid GET verification token      | <code>src/webhook/verifyToken.ts</code> returns 403.                                                                                                                                                                                                     |
| Duplicate webhook (same message id) | <code>src/memory/webhookEventRepo.ts</code> 's <code>claimWebhookEvent()</code> returns false; the router logs "duplicate webhook delivery ignored" and does nothing further.                                                                            |
| AI (OpenRouter) call fails          | Caught in <code>src/pipeline/processInboundMessage.ts</code> ; the customer receives <code>GENERIC_ERROR_REPLY</code> ("Sorry, something went wrong...") and the real error is logged.                                                                   |
| AI requests an unknown tool name    | <code>src/llm/agentLoop.ts</code> returns <code>{"error": "unknown tool: ..."} as the tool result instead of crashing, so the AI can recover gracefully.</code>                                                                                          |
| Malformed tool call arguments       | JSON parsing of the model's tool arguments is wrapped in <code>try/catch</code> ( <code>safeParseArgs</code> ); a parse failure yields an empty args object rather than throwing.                                                                        |
| Google Calendar API failure         | <code>src/tools/checkAvailability.ts/bookAppointment.ts</code> catch the error and return a structured <code>{"error": "..."} result to the AI, which can then tell the customer honestly rather than guessing.</code>                                   |
| WhatsApp send failure               | Logged with the error detail; does not crash the request — the pipeline has already acknowledged Meta's webhook by this point.                                                                                                                           |
| Database problem                    | <code>GET /readiness</code> would report 503 <code>{"status": "not_ready"}</code> if the DB becomes unreachable; <code>better-sqlite3</code> is synchronous so most errors surface immediately and are caught by the pipeline's <code>try/catch</code> . |

# 8. The Database (SQLite Memory)

NOTEBOOK ANALOGY

Think of the database as the receptionist's notebook. Every page is one type of record: one section for "who has called before," one for "what did we talk about," one for "appointments being arranged," and one for "messages we've already dealt with, so we don't answer them twice."

## Schema diagram (actual tables, from `src/memory/migrations/001_init.sql`)



## Table-by-table explanation

| Table                 | Purpose                                                                                    | Why it exists                                                                                                                                                   |
|-----------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| customers             | One row per WhatsApp number that has ever messaged.                                        | Gives every customer a stable identity without any signup — the phone number is the identity.                                                                   |
| conversations         | One row per "session" of memory. status is active or ended.                                | Lets "restart" cleanly start a new memory session without deleting the old one — a customer can have many conversations over time, but only one active at once. |
| conversation_messages | Every user/assistant/tool message, in order.                                               | This is what actually gets sent back to the AI as "conversation history" on the next turn — it's the AI's real memory.                                          |
| booking_sessions      | Partial appointment details (service, date, time, name) collected across several messages. | A booking often can't be completed in one message; this lets the system remember "we already know the                                                           |

service and date, we just need the time" between turns.

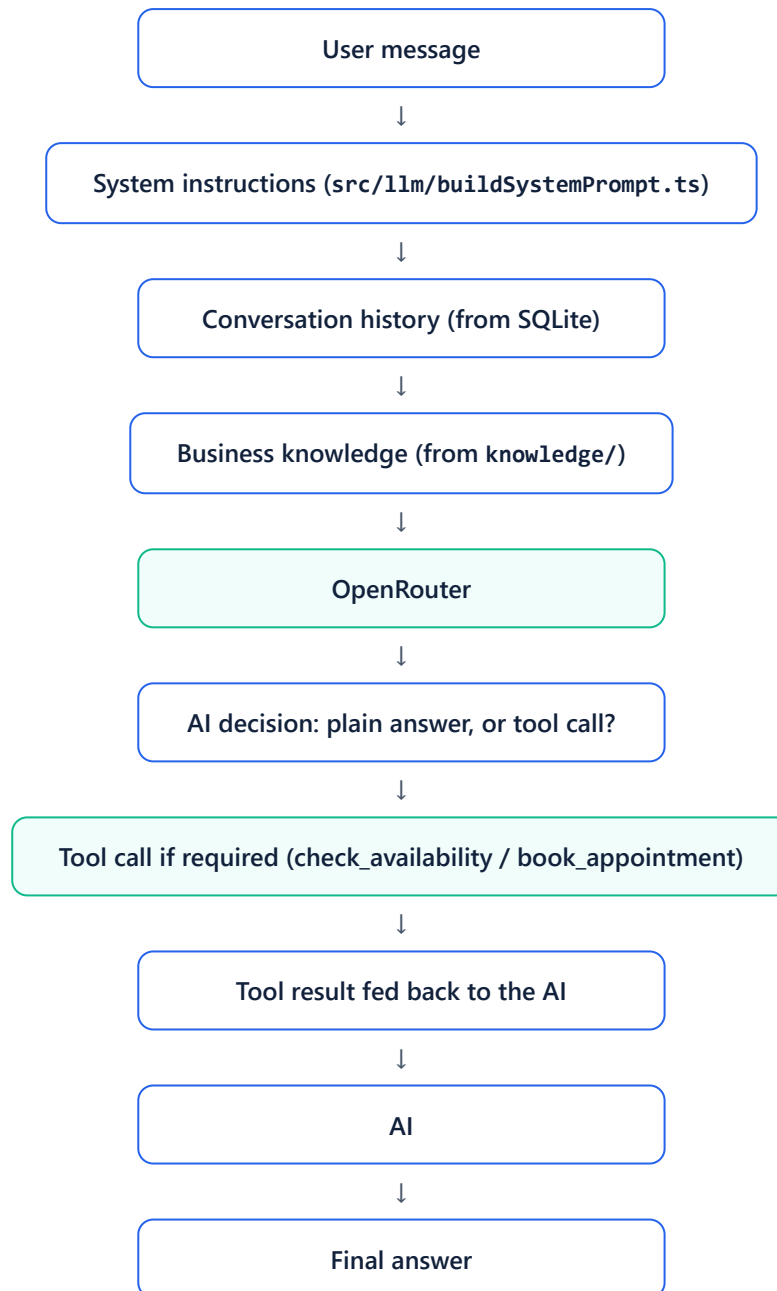
|                |                                             |                                                                                                                                                          |
|----------------|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| webhook_events | Every WhatsApp message ID Meta has sent us. | The primary key is the message ID itself, with INSERT OR IGNORE — this is the exact mechanism that makes duplicate delivery impossible to process twice. |
|----------------|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|

## Persistence & duplicate protection

- The database is a single file (data/app.db), stored outside the Docker container on the VPS disk (a bind-mounted volume) — restarting or rebuilding the container does not lose data.
- PRAGMA journal\_mode = WAL and PRAGMA foreign\_keys = ON are set at connection time (src/memory/db.ts) for durability and referential integrity.
- Restart ("reset") never deletes rows — it flips conversations.status to ended and starts a new row. Nothing is destroyed.

## 9. The AI Agent Loop

---



### The bounded tool loop

The loop in `src/llm/agentLoop.ts` can repeat — the AI can call a tool, look at the result, and decide it needs to call *another* tool before answering. To stop this from running forever (a malfunctioning model looping endlessly, burning cost), the loop is capped at `MAX_TOOL_ROUNDS`, which defaults to **4** (confirmed in `src/config/env.ts`). If the cap is reached without a final plain-text answer, the loop returns a safe fallback message ("I need a moment to look into that properly...") rather than leaving the customer without any reply.

### Safe fallback behavior

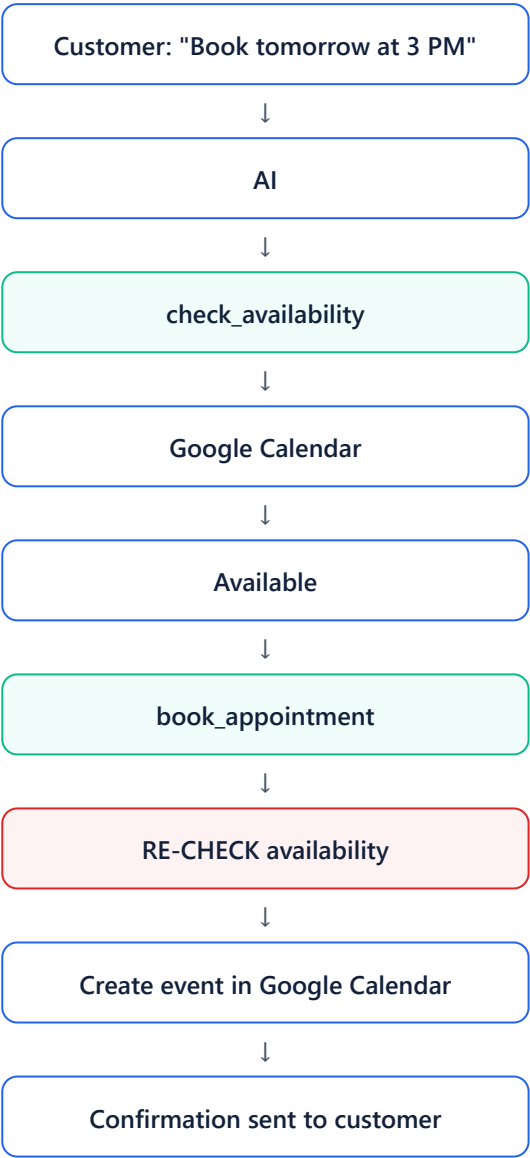
- Unknown tool name requested by the model → returns a structured error to the model, not a crash.
- Malformed tool-call JSON arguments → parsed defensively; falls back to an empty object rather than throwing.

- A tool handler throws an exception → caught, logged, and reported back to the model as `{"error": "tool_execution_failed"}` so the conversation can continue.
- OpenRouter itself is unreachable/errors → caught one level up in the pipeline; customer gets the generic error reply (Chapter 7).

All of this is exercised by `tests/unit/llm/agentLoop.test.ts` with a mocked OpenRouter client.

# 10. The Booking System

## Booking sequence diagram



## What happens if the slot becomes unavailable?

If the re-check (immediately before `events.insert`) finds the slot is no longer free, `src/calendar/booking.ts` returns `{"success": false, "conflict": true}` instead of creating anything. The AI is instructed (system prompt) to apologize and offer alternative times rather than claiming success — it is structurally impossible for the code to report a booking succeeded when Google Calendar didn't actually confirm it.

## Configuration that shapes booking behavior

| Setting                        | Meaning                                                    | Default (if unset) |
|--------------------------------|------------------------------------------------------------|--------------------|
| BUSINESS_TIMEZONE              | IANA timezone name used for all availability/booking math. | UTC                |
| BUSINESS_HOURS_START /<br>_END | Daily working hours considered for free slots.             | 09:00 / 18:00      |

|                          |                                                                      |                            |
|--------------------------|----------------------------------------------------------------------|----------------------------|
| BUSINESS_DAYS            | Which weekdays (1=Mon..7=Sun) the business is open.                  | 1, 2, 3, 4, 5, 6 (Mon–Sat) |
| BOOKING_DURATION_MINUTES | Default appointment length if the AI/customer doesn't specify one.   | 30                         |
| BOOKING_BUFFER_MINUTES   | Extra padding kept around existing events when computing free slots. | 0                          |
| GOOGLE_CALENDAR_ID       | Which calendar is checked/booked.                                    | primary                    |

## Google Calendar access requirements

- A Google Cloud **service account** is used (server-to-server auth, no interactive login).
- The target calendar must be explicitly **shared** with that service account's email, with "**Make changes to events**" permission — otherwise every calendar call will fail with a permissions error.
- Availability is computed via `freebusy.query` (the current, non-deprecated Calendar API v3 endpoint) — see `src/calendar/availability.ts`.

### REQUIRES CONFIGURATION

The calendar integration is fully implemented and unit-tested with a mocked Google API, but it has not been exercised against a real Google Calendar in this environment — that requires real `GOOGLE_CLIENT_EMAIL/GOOGLE_PRIVATE_KEY/GOOGLE_CALENDAR_ID` credentials, which were not available during development.

# 11. Configuration Reference

Every variable below exists in `.env.example` and is validated at startup by `src/config/env.ts` — the app refuses to start if a required one is missing. No real secret values are shown anywhere in this document.

## WhatsApp

| Variable                 | Purpose                                                  | Example | Required           |
|--------------------------|----------------------------------------------------------|---------|--------------------|
| WHATSAPP_ACCESS_TOKEN    | Graph API auth token                                     | *****   | Yes                |
| WHATSAPP_PHONE_NUMBER_ID | Which business phone number to send/receive as           | *****   | Yes                |
| WHATSAPP_VERIFY_TOKEN    | Chosen by you; must match the Meta webhook config screen | *****   | Yes                |
| META_APP_SECRET          | Used to validate X-Hub-Signature-256                     | *****   | Yes                |
| WHATSAPP_API_VERSION     | Graph API version string                                 | v21.0   | No — default shown |

## AI (OpenRouter)

| Variable              | Purpose                                           | Example             | Required           |
|-----------------------|---------------------------------------------------|---------------------|--------------------|
| OPENROUTER_API_KEY    | Auth key for OpenRouter                           | *****               | Yes                |
| OPENROUTER_MODEL      | Which AI model to use — swappable, no code change | openai/gpt-4o-mini  | No — default shown |
| OPENROUTER_SITE_URL   | Sent as HTTP-Referer; optional attribution        | https://example.com | No                 |
| OPENROUTER_APP_NAME   | Sent as X-Title                                   | WhatsApp AI Agent   | No                 |
| OPENROUTER_TIMEOUT_MS | Max wait time for a completion call               | 30000               | No — default shown |

## Google Calendar

| Variable            | Purpose                               | Example | Required           |
|---------------------|---------------------------------------|---------|--------------------|
| GOOGLE_CLIENT_EMAIL | Service account email                 | *****   | Yes                |
| GOOGLE_PRIVATE_KEY  | Service account private key (PEM)     | *****   | Yes                |
| GOOGLE_CALENDAR_ID  | Which calendar to check/book against  | primary | No — default shown |
| GOOGLE_PROJECT_ID   | GCP project id (informational)        | *****   | No                 |
| GOOGLE_TIMEOUT_MS   | Max wait time for a Calendar API call | 15000   | No — default shown |

## Business

| Variable                        | Purpose                             | Example          | Required           |
|---------------------------------|-------------------------------------|------------------|--------------------|
| BUSINESS_NAME                   | Used in the AI's system prompt      | Example Business | No — default shown |
| BUSINESS_TIMEZONE               | IANA timezone for all booking math  | UTC              | No — default shown |
| BUSINESS_PHONE / BUSINESS_EMAIL | Given to customers who need a human | +10000000000     | No                 |
| BUSINESS_HOURS_START / _END     | Daily open hours                    | 09:00 / 18:00    | No — default shown |
| BUSINESS_DAYS                   | Open weekdays, 1(Mon)–7(Sun)        | 1,2,3,4,5,6      | No — default shown |

## Booking

| Variable                 | Purpose                        | Example | Required           |
|--------------------------|--------------------------------|---------|--------------------|
| BOOKING_DURATION_MINUTES | Default appointment length     | 30      | No — default shown |
| BOOKING_BUFFER_MINUTES   | Padding around existing events | 0       | No — default shown |

## Agent behavior

| Variable                   | Purpose                                              | Example                  | Required           |
|----------------------------|------------------------------------------------------|--------------------------|--------------------|
| RESTART_KEYWORDS           | Words that reset a conversation                      | restart,reset,start over | No — default shown |
| CONVERSATION_HISTORY_LIMIT | How many past messages are sent to the AI as context | 20                       | No — default shown |
| MAX_TOOL_ROUNDS            | Cap on the tool-calling loop                         | 4                        | No — default shown |
| WHATSAPP_TIMEOUT_MS        | Max wait time for a WhatsApp send                    | 15000                    | No — default shown |

## Security & Deployment

| Variable              | Purpose                           | Example | Required           |
|-----------------------|-----------------------------------|---------|--------------------|
| MAX_BODY_SIZE         | Max accepted webhook payload size | 1mb     | No — default shown |
| RATE_LIMIT_PER_MINUTE | Requests/min per IP on /webhook   | 120     | No — default shown |

|                           |                                      |                            |                     |
|---------------------------|--------------------------------------|----------------------------|---------------------|
| DATABASE_PATH             | Where the SQLite file lives          | ./data/app.db              | No — default shown  |
| DOMAIN                    | Real domain, used by Caddy for HTTPS | whatsapp-agent.example.com | Yes for deployment  |
| PORT, NODE_ENV, LOG_LEVEL | Standard server configuration        | 3000 / production / info   | No — defaults shown |

# 12. Testing

## EXPLAIN LIKE I'M 10

Tests are like a teacher checking every part of a machine before it's allowed into an exam. Each test builds one tiny, controlled situation (a fake message, a fake calendar answer) and checks that the code reacts exactly the right way — before any real customer ever sees it.

## Verified result (re-run for this documentation)

### VERIFIED · RE-RUN JUST NOW

**19 test files, 87 tests, 87 passed, 0 failed.** Command: `npx vitest run`.

## Test categories

| Category               | Files                                                                                    | What it proves                                                                                                                                       |
|------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Environment validation | <code>tests/unit/config/env.test.ts</code>                                               | Missing required variables fail startup with a clear error.                                                                                          |
| Calendar               | <code>tests/unit/calendar/availability.test.ts</code> ,<br><code>timezone.test.ts</code> | Free-slot computation, business hours/day filtering, buffer logic.                                                                                   |
| Knowledge              | <code>tests/unit/knowledge/loader.test.ts</code>                                         | Markdown/JSON files load correctly into the prompt text.                                                                                             |
| Memory                 | <code>tests/unit/memory/*.test.ts</code> (4 files)                                       | Customers, conversations, messages, booking sessions, and webhook-event idempotency all behave correctly against a real (in-memory) SQLite database. |
| Restart                | <code>tests/unit/restart/isRestartCommand.test.ts</code>                                 | Keyword matching is precise — doesn't misfire mid-sentence.                                                                                          |
| Tools                  | <code>tests/unit/tools/*.test.ts</code> (2 files)                                        | <code>check_availability / book_appointment</code> handle success, conflict, invalid args, and thrown errors — with the real calendar module mocked. |
| LLM agent loop         | <code>tests/unit/llm/agentLoop.test.ts</code>                                            | Plain answers, tool calls, unknown tools, thrown handlers, and the max-rounds fallback — all with OpenRouter mocked.                                 |
| Webhook                | <code>tests/unit/webhook/*.test.ts</code> (3 files)                                      | Signature verification, GET handshake, and payload parsing in isolation.                                                                             |
| Security (integration) | <code>tests/integration/security.e2e.test.ts</code>                                      | Helmet headers present, malformed JSON → 400 not 500, no secret leakage, rate-limit headers present.                                                 |

|                                       |                                                             |                                                                                                                                                                                              |
|---------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Webhook<br>(integration)              | <code>tests/integration/webhook.e2e.test.ts</code>          | Full HTTP round-trip: GET verify, unsigned 401, signed 200, async reply delivery, duplicate no-op.                                                                                           |
| Conversation<br>flow<br>(integration) | <code>tests/integration/conversationFlow.e2e.test.ts</code> | Greeting → knowledge answer → booking → confirmation, booking conflict, restart, concurrent-message locking, and agent-loop failure handling — end to end with all external services mocked. |
| Utilities                             | <code>tests/unit/utls/retry.test.ts</code>                  | Retry/backoff logic and permanent-vs-transient error classification.                                                                                                                         |

All external services (WhatsApp Graph API, OpenRouter, Google Calendar) are mocked in every test — the suite runs with no real credentials and makes no real network calls.

## PART 3

# Running & Deploying

From a developer's laptop to a live VPS answering real WhatsApp messages.

# 13. Running It Locally

| Command                             | What it does                                                                                                       |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>npm install</code>            | Downloads all the code libraries the project depends on.                                                           |
| <code>copy .env.example .env</code> | Creates your own configuration file from the template. Fill in real or test values.                                |
| <code>npm test</code>               | Runs all 87 automated tests. All external services are mocked, so this works without real credentials.             |
| <code>npm run typecheck</code>      | Asks the TypeScript compiler to check every file for type errors, without producing output files.                  |
| <code>npm run lint</code>           | Runs ESLint to catch style/correctness issues.                                                                     |
| <code>npm run build</code>          | Compiles TypeScript into plain JavaScript in <code>dist/</code> , and copies the SQL migration files alongside it. |
| <code>npm run dev</code>            | Starts the server directly from the TypeScript source, restarting automatically on file changes.                   |

## Confirming it's alive

### GET /health

Liveness — "is the process running?" Returns `{"status": "ok"}`.

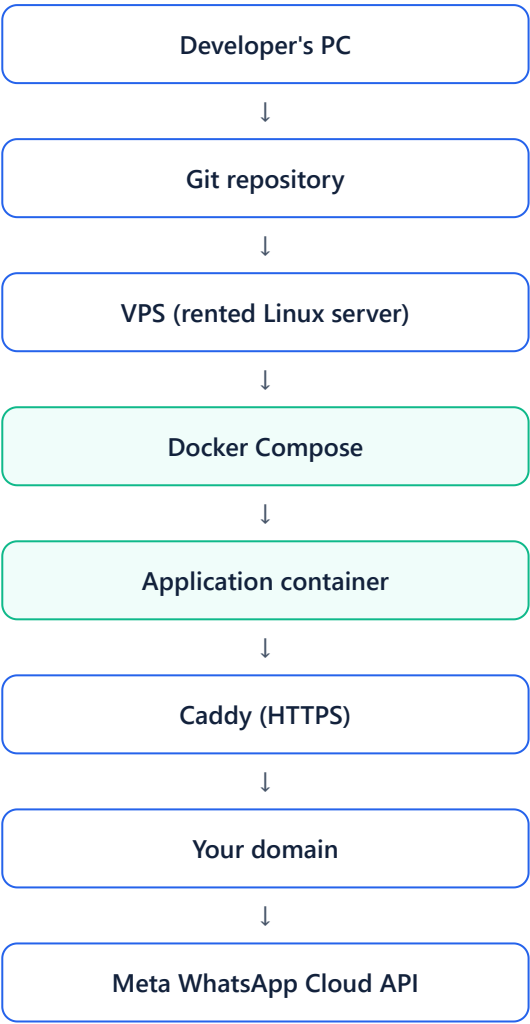
### GET /readiness

Readiness — "can it reach the database?" Returns `{"status": "ready"}`.

### VERIFIED · ACTUAL TERMINAL OUTPUT (previous validation run)

```
$ curl http://localhost:3000/health
{"status": "ok"}
$ curl http://localhost:3000/readiness
{"status": "ready"}
```

# 14. Deploying to a VPS



| Step                                         | Status                                            |
|----------------------------------------------|---------------------------------------------------|
| 1. Domain purchased                          | REQUIRES CONFIGURATION (yours to buy)             |
| 2. DNS A/AAAA record pointed at VPS          | REQUIRES CONFIGURATION                            |
| 3. VPS provisioned with Docker installed     | REQUIRES CONFIGURATION                            |
| 4. .env filled with real production values   | REQUIRES CONFIGURATION                            |
| 5. <code>docker compose build</code>         | NOT VERIFIED IN THIS ENVIRONMENT — see Chapter 21 |
| 6. <code>docker compose up -d</code>         | NOT VERIFIED IN THIS ENVIRONMENT                  |
| 7. Health check reachable over HTTPS         | Depends on steps above                            |
| 8. Readiness check reachable over HTTPS      | Depends on steps above                            |
| 9. Meta webhook configured with the live URL | REQUIRES CONFIGURATION                            |
| 10. OpenRouter key configured                | REQUIRES CONFIGURATION                            |
| 11. Google Calendar configured & shared      | REQUIRES CONFIGURATION                            |

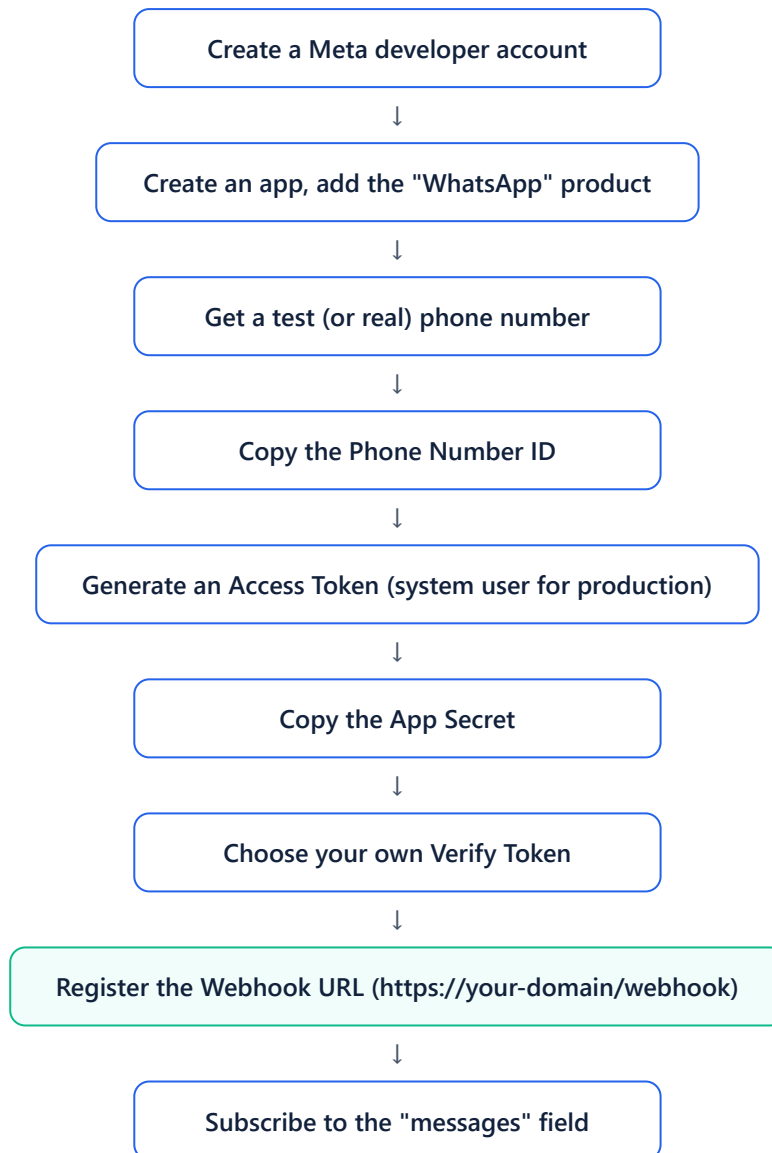
**IMPLEMENTED**

The Dockerfile, docker-compose.yml, and Caddyfile are all written and code-reviewed. The application itself (running directly under Node, outside Docker) has been smoke-tested end-to-end with a real signed HTTP request.

**NOT YET TESTED**

`docker compose build / docker compose up` could not be executed in the development sandbox used to build this project (see Chapter 21/22 for details). Run these yourself on the real VPS before the first live deployment.

## 15. Setting Up Meta WhatsApp Cloud API



| Term            | Plain explanation                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phone Number ID | An internal ID Meta uses to identify which business number is sending/receiving — not the phone number itself.                                                      |
| Access Token    | A password-like string that proves our app is allowed to act as that phone number. Use a permanent "system user" token, not the 24-hour test token, for production. |
| App Secret      | A shared secret used to prove webhook messages really came from Meta (Chapter 6).                                                                                   |
| Verify Token    | A string you invent yourself, entered in both your .env and Meta's webhook config screen, used only for the one-time setup handshake.                               |
| Webhook URL     | Must be https://<your-domain>/webhook — Meta will not accept a plain HTTP URL.                                                                                      |

### DO NOT INCLUDE REAL CREDENTIALS

None of the values above are shown anywhere in this document — only their names and purpose.

## 16. Setting Up OpenRouter

---

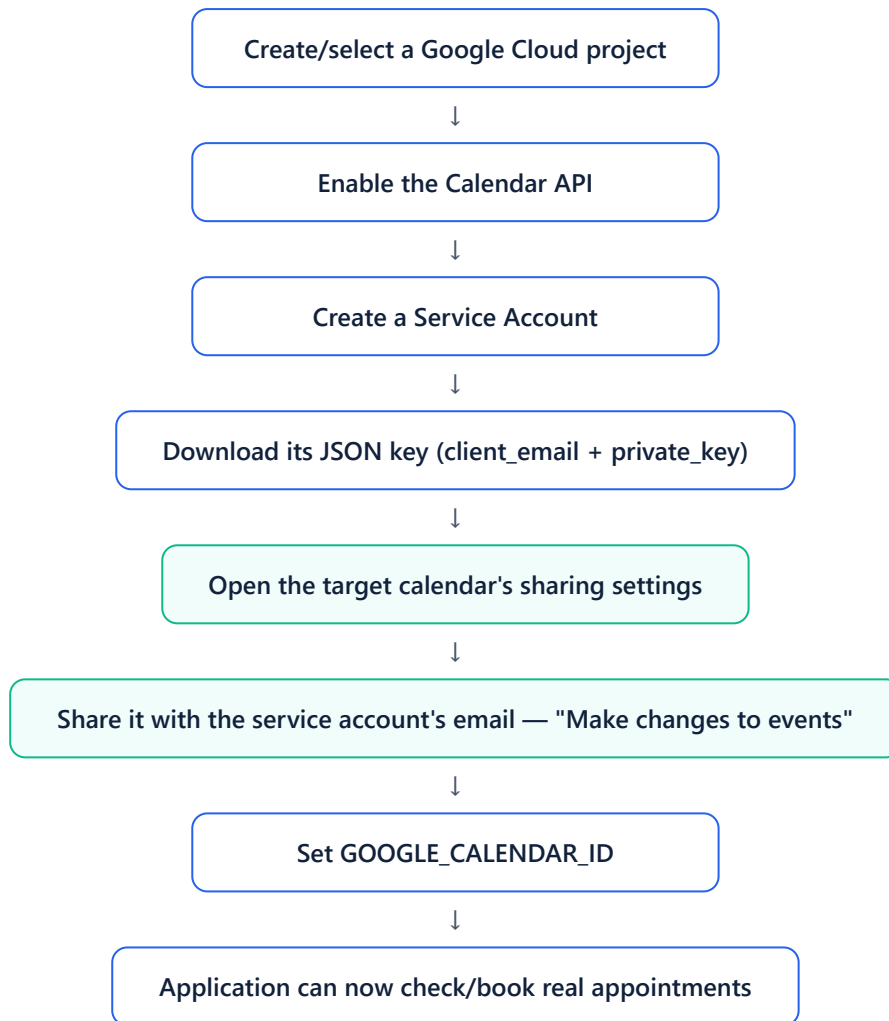
Create an account → Generate an API key → Put it in `OPENROUTER_API_KEY` →

Choose a model in `OPENROUTER_MODEL` → The app uses it automatically

Create an account and key at **[openrouter.ai/keys](https://openrouter.ai/keys)**. The model is set purely by an environment variable — swapping to a different model (e.g. a different provider entirely) never requires touching application code, only redeploying with a new `OPENROUTER_MODEL` value. No API key is shown in this document.

## 17. Setting Up Google Calendar

---



### WHY A SERVICE ACCOUNT?

A service account is like giving the receptionist their own staff badge to the appointment diary — it doesn't need a real person to log in every time; it's a permanent, dedicated identity just for this program.

### REQUIRES CONFIGURATION

This project ships with no real Google credentials — the integration is fully coded and unit-tested with a mocked API, but has not been exercised against a live calendar.

## 18. Managing Business Knowledge

A business owner (no coding needed) can edit these files directly:

| File                   | What to put in it                       |
|------------------------|-----------------------------------------|
| knowledge/business.md  | Name, address, contact details, hours.  |
| knowledge/services.md  | Services and prices.                    |
| knowledge/policies.md  | Cancellation, payment, and other rules. |
| knowledge/faq.md       | Common questions and answers.           |
| knowledge/booking.json | Which services require booking.         |

### Common questions

#### "What happens if I change my opening hours in `business.md`?"

The *text* the AI talks about updates immediately after a restart/redeploy — but the actual **booking math** (which hours count as available) is driven separately by the `BUSINESS_HOURS_START/_END` environment variables, not by this file. Change both together, or the AI's description and the real calendar behavior will disagree.

#### "What happens if I add a new service to `services.md`?"

The AI can immediately describe and quote it after the next restart (knowledge files are loaded once at boot, per `src/knowledge/loader.ts`) — no code change needed. There is no code-level list of "valid services"; the AI relies entirely on this file.

#### "What happens if I change a policy?"

Same as above — edit `policies.md`, restart/redeploy the app, and the new wording is what the AI will use going forward.

#### RESTART REQUIRED

Because knowledge files are only read once at process startup, any edit requires restarting the app (or a fresh deploy) to take effect — simply saving the file while the app is already running has no immediate effect.

## 19. Project File & Folder Map

```
src/  
  config/    env.ts, constants.ts — configuration & validated env vars  
  webhook/   router.ts, verifySignature.ts, verifyToken.ts, parseInboundPayload.ts  
  whatsapp/  client.ts, types.ts — Graph API send calls  
  pipeline/  processInboundMessage.ts, idempotency.ts — orchestration + locking  
  memory/    db.ts, migrations/, customerRepo.ts, conversationRepo.ts,  
             bookingSessionRepo.ts, webhookEventRepo.ts, runMigrations.ts  
  llm/       openRouterClient.ts, buildSystemPrompt.ts, agentLoop.ts, types.ts  
  tools/     checkAvailability.ts, bookAppointment.ts, index.ts (registry)  
  calendar/  googleClient.ts, availability.ts, booking.ts, timezone.ts  
  knowledge/ loader.ts — reads knowledge/*.md,*json into the system prompt  
  restart/   isRestartCommand.ts  
  health/    router.ts — /health, /readiness  
  utils/     retry.ts — backoff + timeout helpers  
  app.ts, index.ts, logger.ts  
tests/  
  unit/      calendar/, config/, knowledge/, llm/, memory/, restart/, tools/, utils/, webhook/  
  integration/ webhook.e2e.test.ts, conversationFlow.e2e.test.ts, security.e2e.test.ts  
  fixtures/knowledge/  
knowledge/   business.md, services.md, policies.md, faq.md, booking.json  
nginx/       Caddyfile  
scripts/     copyAssets.js — copies .sql migrations into dist/ at build time  
data/        .gitkeep (SQLite file lives here at runtime, gitignored)  
secrets/     .gitkeep (mount point for a Google key file, if used that way)  
Dockerfile, docker-compose.yml, .dockerignore  
package.json, package-lock.json, tsconfig.json, vitest.config.ts, .eslintrc.json  
.env.example, .gitignore  
README.md, FINAL_STATUS.md, PRODUCTION_CHECKLIST.md
```

This tree is taken directly from `git ls-files` in the actual repository — nothing here is invented.

## 20. Monitoring & Maintenance

| Task                       | How                                                                                           |
|----------------------------|-----------------------------------------------------------------------------------------------|
| View logs                  | <code>docker compose logs -f app</code> — structured JSON, secrets redacted.                  |
| Check health               | <code>curl https://&lt;domain&gt;/health</code>                                               |
| Check readiness            | <code>curl https://&lt;domain&gt;/readiness</code>                                            |
| Backup the database        | Copy <code>./data/app.db</code> from the host (it's bind-mounted, always on disk).            |
| Restart the service        | <code>docker compose restart app</code>                                                       |
| Update the application     | <code>git pull &amp;&amp; docker compose build app &amp;&amp; docker compose up -d app</code> |
| Dependency/security review | <code>npm audit</code> — see Chapter 22 for the current findings.                             |

### MAINTENANCE CHECKLIST

- ☐ Logs reviewed weekly for repeated errors
- ☐ Database backed up on a schedule
- ☐ `npm audit` re-run after any dependency change
- ☐ Knowledge files reviewed whenever prices/hours/policies change
- ☐ Health/readiness endpoints checked after every deploy

## PART 4

# Status & Reference

Where things really stand today, what's left, and a glossary for every term used in this book.

## 21. Current Project Status Dashboard

Every row below was verified again while writing this documentation, not carried over blindly from an earlier report.

| Area                            | Status                             | Evidence                                                                                                                                                        |
|---------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Application code                | ✓ COMPLETE                         | All planned modules implemented (see Chapter 19 file map).                                                                                                      |
| Tests                           | ✓ COMPLETE                         | 19 files / 87 tests / 87 passed, re-run just now.                                                                                                               |
| Typecheck                       | ✓ COMPLETE                         | <code>npx tsc --noEmit</code> — 0 errors, re-run just now.                                                                                                      |
| Lint                            | ✓ COMPLETE                         | <code>npx eslint . --ext .ts</code> — 0 errors/warnings, re-run just now.                                                                                       |
| Production build                | ✓ COMPLETE                         | <code>npm run build</code> — succeeds, re-run just now.                                                                                                         |
| Security hardening (code-level) | ✓ COMPLETE                         | Helmet, rate limiting, signature verification, secret redaction all implemented and tested.                                                                     |
| Docker build/run                | ● NOT VERIFIED IN THIS ENVIRONMENT | Docker Desktop's engine did not come online in this sandbox after repeated attempts. <b>Run <code>docker compose build</code> yourself before first deploy.</b> |
| Meta WhatsApp Cloud API         | ● REQUIRES CONFIGURATION           | Code implemented and smoke-tested with a real signed request; no real Meta app/credentials configured.                                                          |
| OpenRouter                      | ● REQUIRES CONFIGURATION           | Code implemented and unit-tested; no real API key configured.                                                                                                   |
| Google Calendar                 | ● REQUIRES CONFIGURATION           | Code implemented and unit-tested with a mocked API; no real service account/calendar configured.                                                                |
| Domain                          | ● REQUIRES CONFIGURATION           | Not yet chosen/pointed at a VPS.                                                                                                                                |
| HTTPS                           | ● REQUIRES CONFIGURATION           | Caddy config is written and ready; depends on Domain + VPS.                                                                                                     |
| Business knowledge              | ● REQUIRES CONFIGURATION           | Files exist and load correctly, but contain example/placeholder content only.                                                                                   |
| Real WhatsApp end-to-end test   | ● NOT COMPLETE                     | Cannot happen without a live Meta app and deployment.                                                                                                           |

## 22. Limitations & Risks

`npm audit` findings (re-run for this documentation)

**RE-VERIFIED, NOT ASSUMED**

**12 advisories total: 10 moderate, 1 high, 1 critical.**

| Package                                          | Severity                       | Runtime or dev-only?         | Notes                                                                                                                                                                                                                                                                 |
|--------------------------------------------------|--------------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| vitest, @vitest/mocker, vite, vite-node, esbuild | 1 critical, 1 high, 2 moderate | Dev-only                     | Test-tooling dependency chain. Never included in the production dist//Docker image. Risk to the running application is effectively zero.                                                                                                                              |
| express, body-parser, qs                         | Moderate (x3)                  | Runtime (direct: express)    | Relates to a qs parsing edge case (array-limit bypass / DoS via crafted input). <code>npm audit fix</code> (non-breaking) did not fully resolve this without a further version bump.                                                                                  |
| googleapis, googleapis-common, gaxios, uuid      | Moderate (x4)                  | Runtime (direct: googleapis) | A fix requires a major googleapis version bump (breaking change per npm's own analysis). Not applied, because there are no real Google Calendar credentials in this environment to verify the calendar integration still works correctly against a new major version. |

**RECOMMENDATION**

Before going to production: (1) re-run `npm audit` to confirm nothing has changed, (2) test `npm audit fix --force` for `googleapis` in a branch, then **manually verify** `check_availability` and `book_appointment` against a real test calendar before merging, since this is a breaking major-version change. Do not apply it blindly.

### Other known limitations

- **Docker build was never executed** in the environment this project was built in — the Dockerfile/compose files are manually reviewed for correctness but not proven by an actual build.
- **Graceful shutdown (SIGTERM)** was verified by code review, not by a live signal test, because Windows (the development machine) does not deliver POSIX signals the way the Linux Docker container on a real VPS will.
- **No CI pipeline** is configured yet — `npm test/typecheck/lint/build` are ready to plug into one.
- **No real external integration has been tested** — WhatsApp, OpenRouter, and Google Calendar are all implemented against their documented APIs and unit-tested with mocks, but none has been exercised with real production credentials.

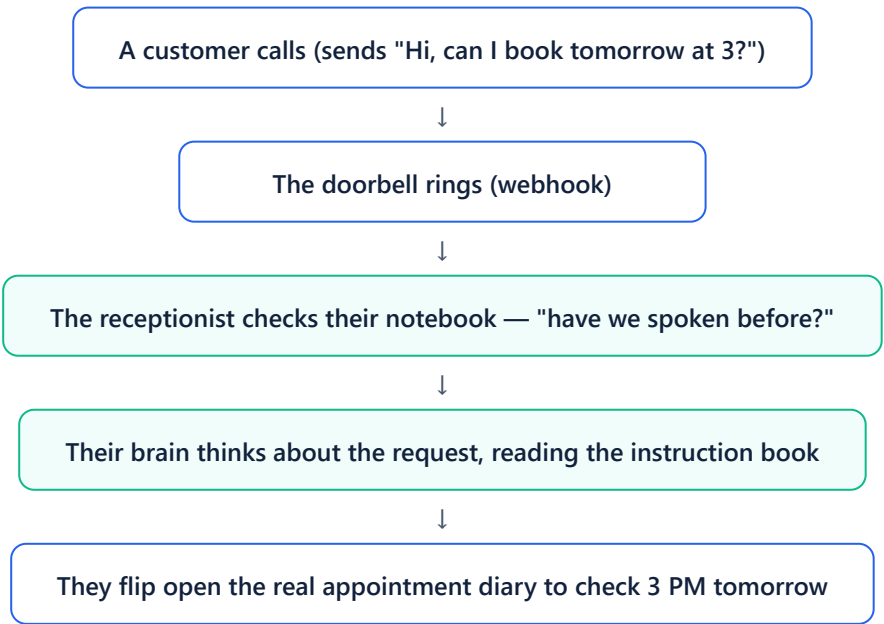
# 23. Explain This Project to a 10-Year-Old

## THE RESTAURANT RECEPTIONIST

Imagine a restaurant with a super-smart receptionist who never sleeps.

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| WhatsApp        | the telephone customers call on                                                    |
| Meta            | the telephone company that connects the call                                       |
| Webhook         | the doorbell that rings the moment someone calls                                   |
| Node.js         | the receptionist themselves                                                        |
| AI (OpenRouter) | the receptionist's brain                                                           |
| Knowledge files | the instruction book on the desk (prices, hours, rules)                            |
| SQLite          | the notebook where they write down every caller                                    |
| Google Calendar | the appointment diary                                                              |
| Tools           | the receptionist's special abilities — like actually flipping open the diary       |
| Docker          | a travel box that lets the whole desk move to any building and still work the same |
| Caddy           | the security guard checking IDs at the door                                        |
| HTTPS           | a locked tunnel nobody else can peek into                                          |
| VPS             | the building where the receptionist's desk actually sits                           |

## One full interaction, beginning to end





It's free! They write it down in the diary



They call the customer back: "You're all set for 3 PM tomorrow!"

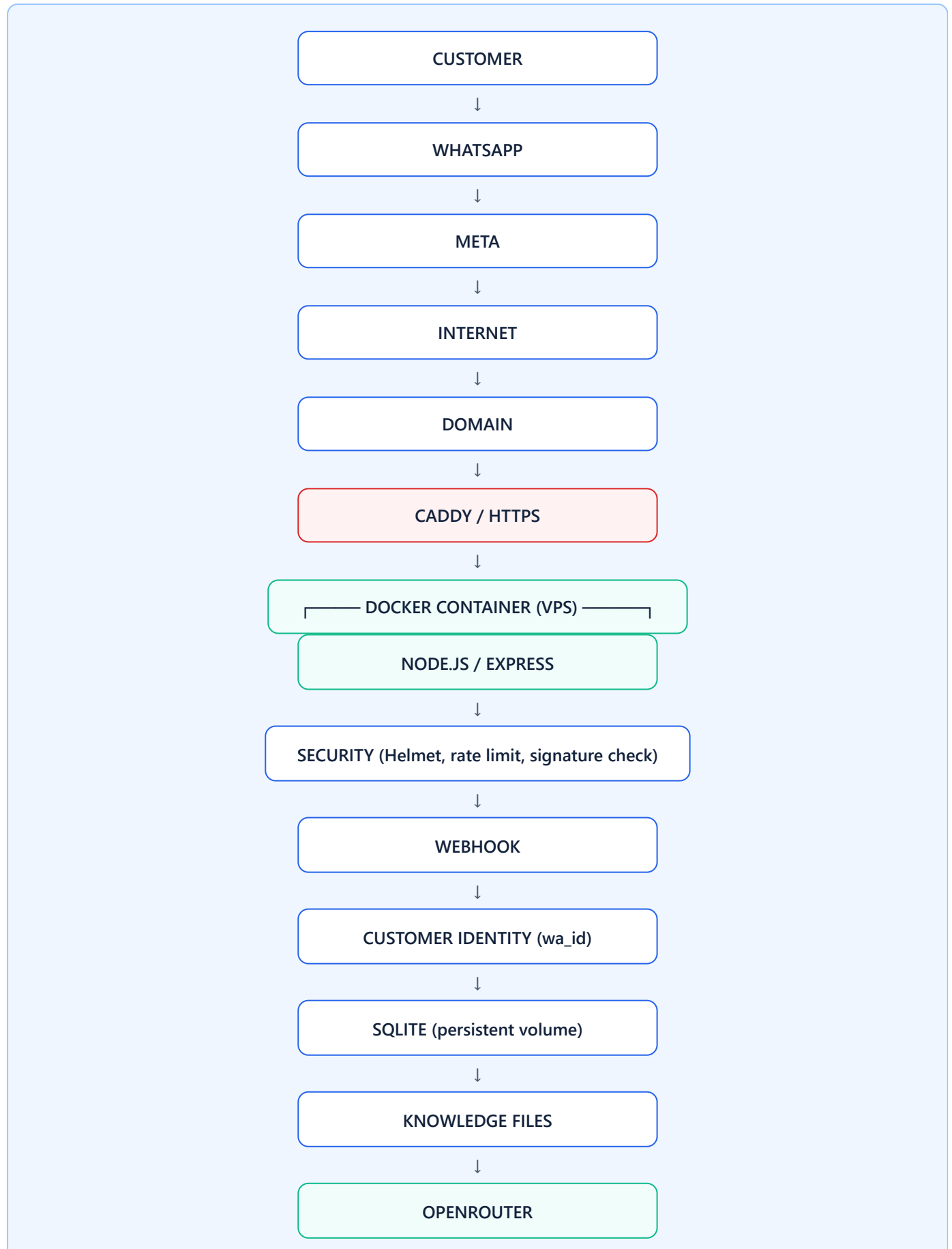
## 24. Glossary

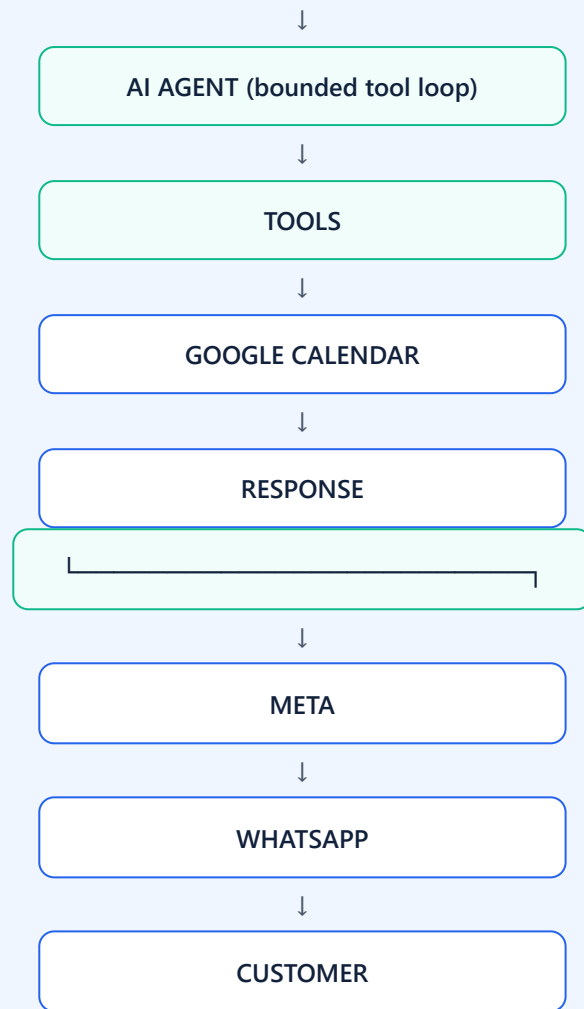
| Term            | Simple meaning                                                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------|
| AI              | A computer program that can understand language and make decisions, like a very fast reader and thinker.                          |
| LLM             | "Large Language Model" — the specific type of AI used here, trained to read and write natural language.                           |
| API             | A way for two computer programs to talk to each other using an agreed set of rules.                                               |
| Webhook         | A "doorbell" — a URL another service calls automatically the moment something happens.                                            |
| HTTP            | The basic language web browsers and servers use to talk to each other.                                                            |
| HTTPS           | HTTP, but locked inside an encrypted tunnel so nobody can eavesdrop.                                                              |
| JSON            | A simple text format for structured data, like <code>{"name": "Alice"}</code> .                                                   |
| HMAC            | A mathematical way to "sign" a message with a secret key, so the receiver can prove it wasn't tampered with.                      |
| SQLite          | A database that lives in a single file, no separate server needed.                                                                |
| SQL             | The language used to ask a database questions ("give me all customers named Alice").                                              |
| Docker          | A tool that packages an app so it runs the same way on any computer.                                                              |
| Container       | The "sealed box" Docker creates and runs.                                                                                         |
| VPS             | "Virtual Private Server" — a rented computer, always on, somewhere on the internet.                                               |
| DNS             | The internet's phonebook — turns a domain name into a computer's real address.                                                    |
| Domain          | A human-readable website address, like <code>example.com</code> .                                                                 |
| SSL/TLS         | The technology behind HTTPS's encryption.                                                                                         |
| OAuth           | A standard way to let one app access another on a user's behalf, without sharing a password directly.                             |
| Service Account | A permanent, non-human identity a program uses to access another service (used here for Google Calendar).                         |
| OpenRouter      | A single service that connects to many different AI models.                                                                       |
| Meta Cloud API  | The official service from Meta (WhatsApp's parent company) for sending/receiving WhatsApp messages programmatically.              |
| wa_id           | The unique WhatsApp identifier for a customer (essentially their phone number).                                                   |
| Idempotency     | Doing the same operation twice has the same effect as doing it once — used here to stop duplicate messages being processed twice. |
| Concurrency     | Multiple things happening (or trying to happen) at the same time.                                                                 |

|                      |                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------|
| Rate Limiting        | Capping how many requests a single source can make in a given time window.                                          |
| Middleware           | Code that runs on every request before it reaches the final handler (e.g. security headers, logging).               |
| Endpoint             | A specific URL a server responds to, like <code>/webhook</code> or <code>/health</code> .                           |
| Environment Variable | A configuration value set outside the code (in <code>.env</code> ), so secrets/settings never live in source files. |
| Tool Calling         | An AI pausing to run a real function (like checking a calendar) instead of guessing an answer.                      |
| Prompt               | The text given to an AI to tell it what to do or say.                                                               |
| System Prompt        | The hidden instructions given to the AI before the conversation starts, defining its role and rules.                |
| Calendar API         | Google's programmatic interface for reading and creating calendar events.                                           |
| REST API             | A common style of web API built around standard HTTP verbs (GET, POST, etc.) and URLs.                              |

## 25. Final Master Diagram

The complete system, top to bottom, in one picture — Docker/VPS wraps the application; everything inside runs in one container.





## 26. Source of Truth & Generation Record

**Documentation generated from the actual project source code** at C:\Websites\WhatsApp AI Agent.

|                                  |                                                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------|
| <b>Generation date:</b>          | 2026-09-10                                                                                |
| <b>Git commit:</b>               | 647193d (2026-09-07) — "fix: remove explicit moduleResolution from tsconfig"              |
| <b>Test result:</b>              | 19 test files, 87 tests, 87 passed (re-run for this document)                             |
| <b>Typecheck / Lint / Build:</b> | All clean (re-run for this document)                                                      |
| <b>Docker verification:</b>      | Not verified in this environment (see Chapter 21/22)                                      |
| <b>Documentation status:</b>     | Reflects the codebase as of the commit above; not a certification of production readiness |

### IMPORTANT

This document is a description of what the code does today, verified against real test runs and a real smoke test. It is **not** proof of production readiness — the WhatsApp, OpenRouter, and Google Calendar integrations still require real credentials and a real end-to-end test before this system should be trusted with real customers.